



Spark **Performance** Tuning & Optimization

A practitioner's guide to making PySpark jobs fast — how Spark executes, diagnosing bottlenecks in the Spark UI, caching, shuffles and joins, memory and cluster sizing, and vectorized libraries. Every technique measured against the **SkyOps Analytics** fleet pipeline, monitored inside Skywise.

6

Modules

10

Diagrams

6

Hands-on labs

UI

Driven tuning

What you will learn

Six modules that build a disciplined, measurement-first approach to Spark performance — each with detailed notes, diagrams, real Spark UI walkthroughs, a SkyOps example tuned before and after, and a hands-on lab. Assumes the PySpark Fundamentals handbook as background.

- | | | |
|--|---|-----------|
| 1 | How Spark Executes: Catalyst, Tungsten & the DAG | M1 |
| Lazy evaluation · stages & shuffles · the optimizer & execution engine | | |
| <hr/> | | |
| 2 | Diagnosing Bottlenecks with the Spark UI | M2 |
| Reading Jobs/Stages/Executors · skew, spill & GC · monitoring in Skywise | | |
| <hr/> | | |
| 3 | Caching & Persistence Strategies | M3 |
| When to cache · storage levels · releasing memory | | |
| <hr/> | | |
| 4 | Shuffle, Partitions & Joins | M4 |
| Shuffle cost · partitions & AQE · broadcast joins · skew | | |
| <hr/> | | |
| 5 | Memory, Resources & Cluster Configuration | M5 |
| Executor memory · spill/OOM · resource allocation · autoscaling | | |
| <hr/> | | |
| 6 | Integrating Optimized Libraries & UDFs | M6 |
| Pandas UDFs & Arrow · NumPy/Pandas · ML · vectorized operations | | |
| <hr/> | | |

01

FOUNDATIONS

How Spark Executes: Catalyst, Tungsten & the DAG

You cannot tune what you do not understand. This module is the execution model every later optimisation builds on.

How Spark Executes: Catalyst, Tungsten & the DAG

Performance tuning is mostly about removing work and moving less data. To know where the work and the data movement are, you need a clear picture of how Spark turns your **SkyOps** DataFrame code into tasks on executors.

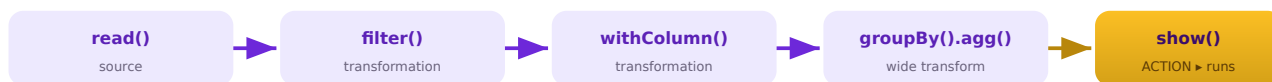
🎯 Learning objectives

- Explain lazy evaluation and the job → stage → task hierarchy
- Identify narrow vs wide transformations and stage boundaries
- Describe what the Catalyst optimizer does for you
- Explain the Tungsten execution engine
- Read a physical plan with explain()
- Locate where time and data movement actually go

1.1 Lazy evaluation and the DAG

Transformations (`select`, `filter`, `join`) are **lazy** — they build a **logical plan**, not results. An **action** (`count`, `write`, `collect`) triggers execution. Spark compiles the plan into a **DAG** and splits it into **stages** at every **shuffle**; each stage runs as parallel **tasks**, one per partition.

Lazy evaluation — transformations build a DAG, action triggers execution



Nothing computes here — Spark only records the lineage (logical plan).



📌 Narrow vs wide — the most important distinction

Narrow transformations (`map`, `filter`) keep data on the same partition — cheap, pipelined.

Wide transformations (`groupBy`, `join`, `distinct`) require a **shuffle** — data crosses the network and a new stage begins. Almost every performance problem lives at a shuffle.

1.2 The Catalyst optimizer

Catalyst rewrites your query before it runs. It resolves names against the catalog, applies **rule-based** optimisations (predicate pushdown, column pruning, constant folding, join reordering), enumerates **physical plans**, and uses **cost-based** estimates to pick one. You get all of this for free by writing DataFrame or SQL code rather than RDDs.

Catalyst optimizer & Tungsten — from your code to fast machine work

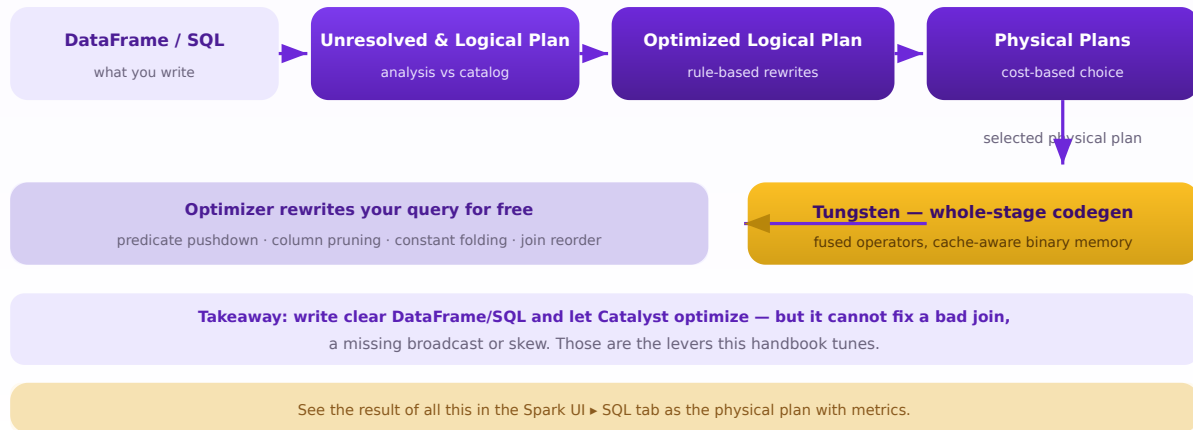


Figure 1.1. Catalyst rewrites your query through logical and physical planning; Tungsten then generates fused, cache-aware code. You get this optimisation for free — but not fixes for bad joins or skew.

✈ Catalyst earns its keep on SkyOps

When we read `telemetry` Parquet and filter to one airline then select three columns, Catalyst **pushes the filter into the scan** and **prunes columns**, so Spark reads a fraction of the files. We did nothing special — but we also cannot rely on it to fix a skewed join.

1.3 The Tungsten engine

Where Catalyst decides *what* to run, **Tungsten** makes it fast. It uses **whole-stage code generation** to fuse a chain of operators into a single tight loop of generated Java, works on a compact **off-heap binary** memory format (no per-row Java objects), and is **cache-aware** to use CPU efficiently. This is why the DataFrame API reaches near-JVM speed — and why dropping into row-by-row Python (Module 6) throws that speed away.

1.4 Read the plan, find the work

`explain()` prints the physical plan. Two things to look for: **Exchange** nodes (each is a shuffle — a stage boundary and a cost) and the scan's **PushedFilters / pruned columns** (proof the optimiser reduced I/O).

```
PYSPARK / PYTHON

busy = (spark.table("skyops_prod.fleet.telemetry")
        .filter("airline = 'SK'")
        .select("aircraft_id", "altitude_ft", "fuel_tph")
        .groupBy("aircraft_id").avg("fuel_tph"))

busy.explain(mode="formatted")  # read this before you tune anything
```

CONSOLE OUTPUT

```
== Physical Plan ==
* HashAggregate (avg)          <- final aggregate
+- Exchange hashpartitioning    <- SHUFFLE (a stage boundary)
  +- * HashAggregate (partial)  <- map-side pre-aggregate
    +- * Project [aircraft_id, fuel_tph]
      +- * Filter (airline = SK)
        +- Scan parquet PushedFilters:[airline=SK] <- I/O reduced
```

💡 One Exchange, one stage boundary

Count the **Exchange** nodes in your plan — each is a shuffle. Fewer, smaller shuffles is the headline goal of almost everything in Modules 3-5.

🔧 Lab 1.1 — Read your own plan

Take a SkyOps query that joins telemetry to the aircraft registry and groups by type. Run `explain(mode="formatted")`. Mark every **Exchange**, confirm the filter was pushed into the Parquet scan, and predict which stage will be the most expensive. Keep this plan — you will tune it in later labs.

🔑 Key takeaways

- Transformations are lazy and build a DAG; an action triggers stages of parallel tasks
- Stages are split at shuffles; narrow = cheap, wide = a network shuffle
- Catalyst optimises your query (pushdown, pruning, join choice) for free when you use DataFrames/SQL
- Tungsten generates fused, off-heap, cache-aware code for near-JVM speed
- `explain()` reveals shuffles (Exchange) and I/O reduction — read it before tuning

02

DIAGNOSIS

Diagnosing Bottlenecks with the Spark UI

Stop guessing. The Spark UI shows exactly which stage is slow and why — skew, spill, shuffle or GC — including from inside Skywise.

SkyOps Analytics · Spark Performance · Module 2 of 6

Diagnosing Bottlenecks with the Spark UI

Optimising without measuring is guesswork. The **Spark UI** is the single source of truth for where a job spends its time. This module is a diagnostic workflow: find the slow stage, read its task metrics, name the cause.

🎯 Learning objectives

- Navigate the Spark UI tabs and what each tells you
- Find the slowest stage from the Jobs and Stages views
- Recognise skew, spill, heavy shuffle and GC signatures
- Use the Executors tab to spot memory and GC pressure
- Access the Spark UI from inside Skywise / the platform
- Apply a repeatable measure-find-fix loop

2.1 A tour of the Spark UI

Every running application exposes a web UI. The tabs you will live in: **Jobs** (each action becomes a job), **Stages** (tasks, shuffle, spill), **SQL/DataFrame** (the physical plan with per-operator metrics), **Storage** (what is cached and how much fits), and **Executors** (memory, cores, GC, task failures).

Spark 3.5 Jobs Stages Storage Environment Executors SQL SkyOpsAnalytics · application_1732_0007					
Completed Jobs (3)					
Job Id	Description	Stages	Tasks	Duration	
2	show at fleet_kpis.py:48	2/2	16/16	0.9 s	
1	groupBy/agg at fleet_kpis.py:40	2/2	208/208	3.4 s	
0	parquet read at fleet_kpis.py:22	1/1	8/8	1.1 s	

Event timeline — green = compute, gold = shuffle write

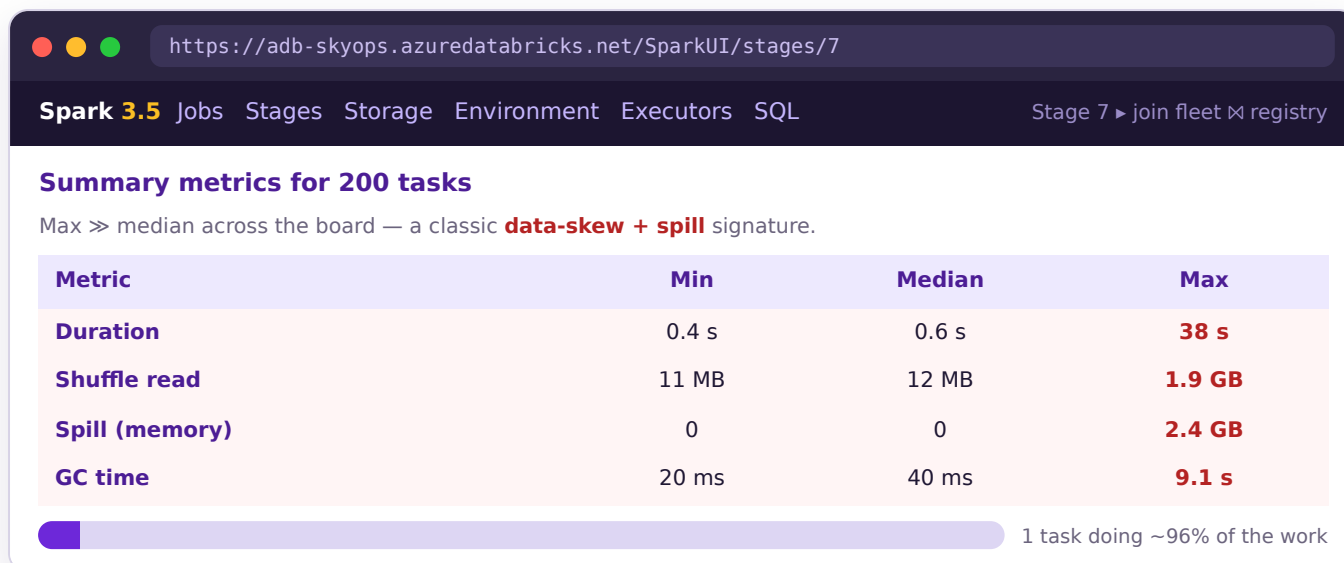
💡 Start at the top, drill down

Open **Jobs**, find the job that took longest, click into its **longest stage**, then read that stage's task summary. Ninety percent of tuning starts by drilling to the one slow stage rather than scanning everything.

2.2 Reading stages: the bottleneck signatures

A stage summarises its tasks as **min / median / max**. The shape of those numbers names the problem:

Signature	What you see	Likely cause
Skew	max duration & shuffle-read $\gg$ median	One key has most of the rows
Spill	non-zero “Spill (memory/disk)”	Partition too big for execution memory
Heavy shuffle	large shuffle read/write bytes	Wide op moving too much data
GC pressure	high GC time vs task time	Too little memory / too many objects
Tiny tasks	thousands of sub-second tasks	Too many partitions, scheduling overhead



Screenshot 2.2. The Stages tab summarises tasks as min / median / max. When the max dwarfs the median for duration, shuffle read and spill, you are looking at data skew — the single most common Spark bottleneck.

✈️ The SkyOps nightly job was slow — here is why

Stage 7 (the registry join) showed a max task of 38 s against a 0.6 s median, 1.9 GB shuffle read on one task, and 2.4 GB spill. That is textbook **skew**: a handful of busy tail numbers carry most rows. The fix lives in Module 4 (broadcast / AQE skew handling).

2.3 The Executors tab: memory & GC

The **Executors** tab shows per-executor memory used vs available, cumulative **GC time**, shuffle bytes and failed tasks. Red flags: GC time that is a large fraction of task time, storage memory pegged at 100%, or executors dying and being replaced.

Spark 3.5 Executors					
Executor ID	Cores	Active Tasks	Storage Memory	Input	Shuffle R/W
driver	—	0	0 / 2.1 GB	0 B	0 / 0
1	4	4	512 MB / 4.6 GB	1.2 GB	320 MB / 410 MB
2	4	4	498 MB / 4.6 GB	1.1 GB	300 MB / 395 MB

2 executors $\times$ 4 cores = 8 parallel tasks. Watch **Shuffle** columns to spot wide-transformation cost.

2.4 Monitoring from inside Skywise

Skywise runs Spark on Palantir Foundry, so you reach the same Spark UI through the platform rather than a public port. From a **Code Workbook** or a build’s **Spark details**, open the Spark UI / driver logs for that run; the tabs and metrics are identical to open-source Spark. On Databricks

the same UI hangs off the cluster or the job run. The skill transfers everywhere — only the door changes.

If you cannot reach a live UI

For a finished run, use the **Spark History Server** (or the platform's run history) to inspect the same stages and task metrics after the fact. Never optimise from the code alone when the measurements are one click away.

2.5 The diagnostic loop

Make tuning a disciplined cycle, changing **one thing at a time**:

(1) Measure — run, open the Spark UI. **(2) Find** — the longest stage. **(3) Diagnose** — read its task summary for a signature above. **(4) Fix** — apply the matching technique from Modules 3-6. **(5) Re-measure** — confirm the stage improved and nothing else regressed.

Lab 2.1 — Find the slow stage

Run the SkyOps nightly query from Lab 1.1. Open the Spark UI, identify the job and its slowest stage, and capture its task **min/median/max** for duration, shuffle read and spill. Write one sentence naming the bottleneck signature, then state which later module fixes it. This is your baseline measurement.

Key takeaways

- The Spark UI is the source of truth — Jobs, Stages, SQL, Storage and Executors
- Drill from the longest job to its longest stage rather than scanning everything
- max $\gg$ median = skew; non-zero spill = memory pressure; large shuffle bytes = move less data
- The Executors tab reveals GC pressure and dying executors
- Inside Skywise/Foundry the same UI is reached via the build's Spark details; the skill transfers
- Tune in a loop: measure, find, diagnose, fix one thing, re-measure

03

REUSE

Caching & Persistence Strategies

Caching turns repeated work into a one-time cost — but only when you reuse data, pick the right storage level, and release it.

Caching & Persistence Strategies

Because transformations are lazy, Spark recomputes a DataFrame's whole lineage every time you call an action on it. When you use the same intermediate result several times, **caching** pays that cost once.

🎯 Learning objectives

- Explain why and when caching helps
- Use `cache()` and `persist()` with the right storage level
- Choose between memory, disk and serialized levels
- Recognise when caching hurts rather than helps
- Release cached data with `unpersist()`
- Know how cache differs from checkpoint and disk cache

3.1 Why cache at all

If a cleaned `silver_fleet` DataFrame feeds three downstream reports, each report re-reads and re-cleans from scratch — three times the work. Cache it after the expensive cleaning step and the next two reports read from memory.

⚠️ The one rule of caching

Cache only data you reuse **at least twice**. Caching something used once wastes memory and can *slow* the job by evicting useful blocks. If in doubt, do not cache.

3.2 `cache()`, `persist()` and storage levels

`cache()` is shorthand for a default `persist()`. For DataFrames the default level is **MEMORY_AND_DISK** (it spills rather than recomputes); `persist(level)` lets you choose. Both are lazy — the data is materialised on the next action.

Persistence storage levels — pick the cheapest that fits

Storage level	Memory	Disk	Serialized	When to use
MEMORY_ONLY	✓	—	no	default <code>cache()</code> ; fastest, may evict
MEMORY_AND_DISK	✓	spill	no	safe default for big cached data
MEMORY_AND_DISK_SER	✓	spill	yes	compact; less RAM, more CPU
DISK_ONLY	—	✓	yes	recompute is costlier than disk read
..._2 (replicated)	✓	✓	—	resilience for very expensive results

Only cache data you reuse $\geq 2\times$. Always `unpersist()` when done — stale cache wastes the storage region.

Figure 3.1. Storage levels trade memory, disk, CPU and resilience. Pick the cheapest level that fits, and only cache data you genuinely reuse.

```

from pyspark import StorageLevel

silver = clean(spark.table("skyops_prod.fleet.telemetry"))
silver.persist(StorageLevel.MEMORY_AND_DISK)  # explicit, safe default
silver.count()                                # action materialises the cache

report_a = silver.groupBy("type").avg("fuel_tph")    # reads from cache
report_b = silver.groupBy("alt_band").count()        # reads from cache

silver.unpersist()    # release when the reports are done

```

💡 Check the Storage tab

After the first action, the Spark UI **Storage** tab shows the cached dataset, its size, and the **fraction cached**. If it is below 100%, blocks are being evicted — use a serialized or disk level, or cache less.

3.3 When caching hurts

Cache is not free: it consumes the same memory region that **execution** needs for shuffles and joins. Over-caching starves execution, causing spill elsewhere. Avoid caching data used once, data you immediately write out, or anything so large it cannot fit — for those, a serialized or **DISK_ONLY** level, or no cache at all, is better.

3.4 Cache vs checkpoint vs disk cache

Mechanism	What it does	Use when
cache / persist	Keeps the result in memory/disk, lineage retained	Reusing a result within a job
checkpoint	Writes to reliable storage and <i>truncates lineage</i>	Very long lineage / iterative jobs
disk cache	Engine caches remote files on local SSD (e.g. Databricks)	Repeated reads of the same Parquet/Delta

✈️ SkyOps caching that paid off

Our feature-engineering notebook builds one `silver_features` table then trains and validates several models against it. Persisting it once (**MEMORY_AND_DISK**) cut the notebook's runtime by roughly a third, because the expensive joins ran a single time.

🔧 Lab 3.1 — Measure a cache

Take a SkyOps query whose cleaned DataFrame feeds two aggregations. Run it uncached and note the total time and the repeated scan in the Spark UI. Add `persist(MEMORY_AND_DISK)` + an action to materialise, re-run, and compare. Check the **Storage** tab for fraction cached, then `unpersist()`.

🔑 Key takeaways

- Spark recomputes lineage per action — cache results you reuse $\geq 2\times$
- `cache()` = default `persist()`; DataFrame default level is `MEMORY_AND_DISK`
- Pick the cheapest storage level that fits; serialized/disk save RAM at some CPU cost
- Over-caching starves execution memory and causes spill — do not cache single-use data
- Always `unpersist()` when done; watch the Storage tab's fraction cached
- checkpoint truncates lineage; disk cache speeds repeated file reads — both differ from cache

04

DATA MOVEMENT

Shuffle, Partitions & Joins

The shuffle is the most expensive thing Spark does. Tuning partitions, broadcasting small tables and taming skew is where the big wins are.

Shuffle, Partitions & Joins

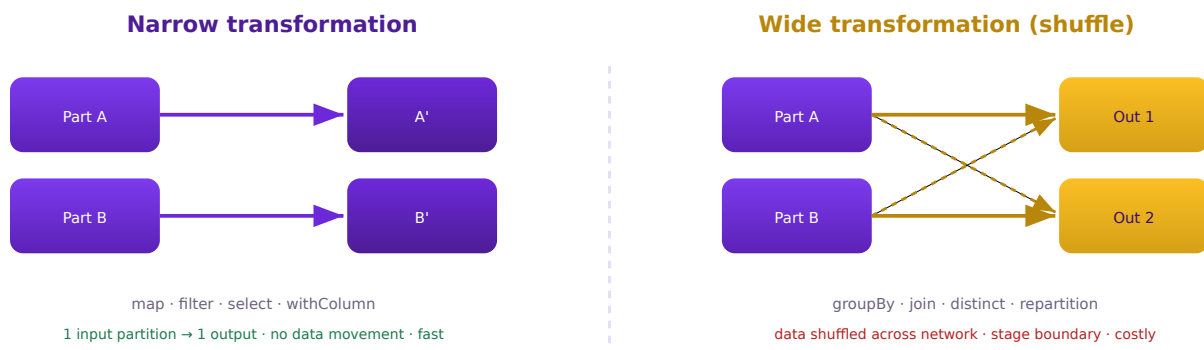
Most slow Spark jobs are slow because of one thing: moving too much data across the network in a **shuffle**. Get shuffles right and everything else is a rounding error.

🎯 Learning objectives

- Explain what a shuffle is and why it is expensive
- Tune `spark.sql.shuffle.partitions` and rely on AQE
- Use broadcast joins to avoid shuffling a large table
- Detect and fix data skew
- Choose repartition vs coalesce correctly
- Cut the number and size of shuffles in a real query

4.1 What a shuffle costs

A **wide** transformation must regroup rows by key across the cluster: every executor writes shuffle files, then every executor reads the slices it needs. That is disk I/O, serialization and network — orders of magnitude slower than in-memory narrow work. The goal is never “no shuffles” (joins need them) but **fewer, smaller, balanced** ones.



4.2 Shuffle partitions & AQE

After a shuffle, the number of output partitions is `spark.sql.shuffle.partitions` (default **200**). Too few → huge partitions that spill; too many → thousands of tiny tasks with scheduling overhead. The old advice was to hand-tune this per job. Today, **Adaptive Query Execution (AQE)** does most of it for you — it coalesces small partitions, switches joins to broadcast, and splits skewed partitions using *runtime* statistics.

Adaptive Query Execution (AQE) — re-optimize using runtime stats

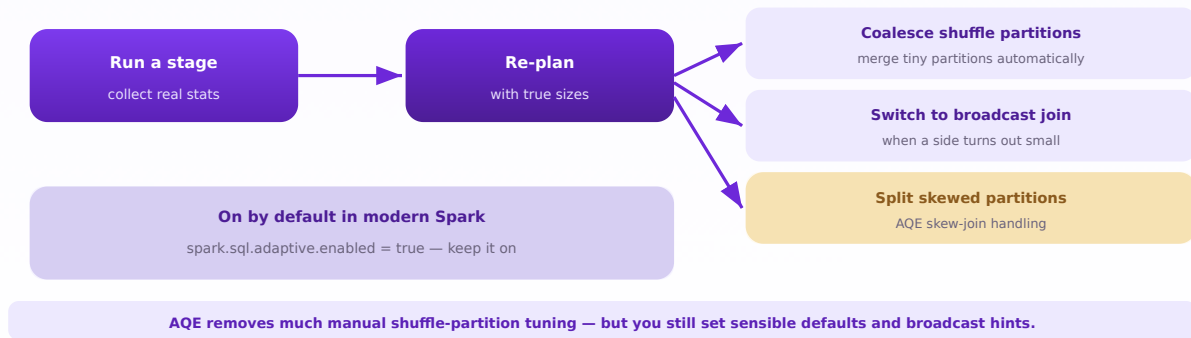


Figure 4.3. Adaptive Query Execution re-plans mid-query using real statistics: coalescing partitions, switching to broadcast, and splitting skewed partitions.

💡 Keep AQE on, set a sane default

Leave `spark.sql.adaptive.enabled=true` (default in modern Spark). Still set `shuffle.partitions` in the right ballpark — aim for partitions of ~128-256 MB after the shuffle — and let AQE fine-tune from there.

4.3 Broadcast joins

When one side of a join is small (a dimension table — our aircraft registry), shuffling both sides is wasteful. A **broadcast join** ships the small table to every executor so the large table never moves.

Broadcast hash join — avoid the shuffle when one side is small

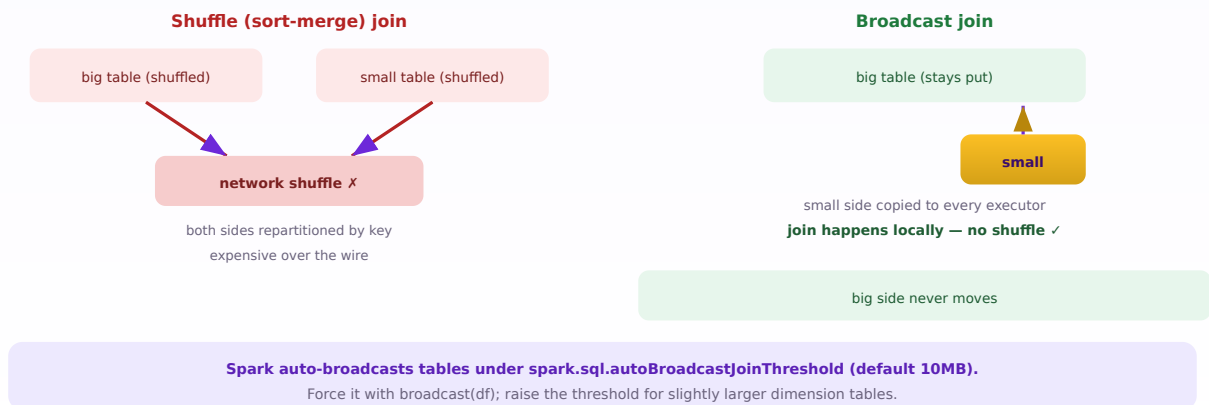


Figure 4.2. A broadcast join copies a small table to every executor so the large table never shuffles — usually far cheaper than a sort-merge join.

```

from pyspark.sql.functions import broadcast

telemetry = spark.table("skyops_prod.fleet.telemetry") # large, partitioned
registry = spark.table("skyops_prod.ref.aircraft") # small dimension

# Force the small side to broadcast – no shuffle of telemetry
fleet = telemetry.join(broadcast(registry), "aircraft_id", "left")

fleet.explain() # look for BroadcastHashJoin (not SortMergeJoin)

```

The threshold

Spark auto-broadcasts tables below `spark.sql.autoBroadcastJoinThreshold` (default **10 MB**). Raise it for slightly larger dimension tables, or use `broadcast()` to force it — but never broadcast something that does not comfortably fit in each executor's memory.

4.4 Taming data skew

Skew is when one key holds most of the rows, so one task does most of the work (the Module 2 signature: $\max \gg \text{median}$). Fixes, in order of preference: let **AQE skew-join** split the hot partition automatically; **broadcast** the other side if it is small; or **salt** the key — add a random suffix to spread the hot key across partitions, then aggregate in two steps.

Fixing the SkyOps registry join

The skewed Stage 7 from Module 2 had a small registry on one side — so we simply `broadcast()` it. The join became a BroadcastHashJoin, the 1.9 GB single-task shuffle vanished, and the stage dropped from 38 s to under 3 s. One line, measured before and after.

4.5 repartition vs coalesce

`repartition(n)` does a **full shuffle** to exactly n balanced partitions — use it to increase parallelism or rebalance skew. `coalesce(n)` **merges** partitions without a full shuffle — use it to *reduce* partitions cheaply, e.g. before writing a handful of output files. Reaching for repartition when coalesce would do adds a needless shuffle.

Lab 4.1 — Halve a query's runtime

Take the skewed join from Lab 2.1. (1) Confirm it is a SortMergeJoin in `explain()`. (2) Wrap the small side in `broadcast()` and confirm it becomes a BroadcastHashJoin. (3) Re-run and compare the stage's max-task time in the Spark UI. (4) Experiment with `shuffle.partitions` and note AQE coalescing in the SQL tab.

🔑 Key takeaways

- Shuffles dominate cost; aim for fewer, smaller, balanced ones — not zero
- `shuffle.partitions` defaults to 200; target ~128-256MB per partition and keep AQE on
- Broadcast small dimension tables to avoid shuffling the large side (threshold 10MB by default)
- Skew = max $\gg$ median; fix with AQE skew-join, broadcast, or salting
- repartition is a full shuffle to rebalance; coalesce merges cheaply to reduce partitions
- Always confirm the join type and stage timing in `explain()` and the Spark UI

05

RESOURCES

Memory, Resources & Cluster Configuration

Executor memory, partition sizing and cluster shape decide whether work fits and scales — or spills, crashes and idles.

Memory, Resources & Cluster Configuration

Throwing nodes at a slow job rarely helps. Performance and stability come from matching **memory**, **cores** and **partitions** to the work — and sizing the cluster to scale with the data, not ahead of it.

🎯 Learning objectives

- Describe the executor memory model
- Diagnose and fix spill and out-of-memory errors
- Allocate executors, cores and memory sensibly
- Size partitions to fit execution memory
- Configure clusters and autoscaling for the workload
- Reason about scalability as data grows

5.1 The executor memory model

Each executor's heap is divided into a small **reserved** slice, **user memory** (your objects and UDF state), and a large **unified** region shared by **execution** (shuffles, joins, sorts, aggregations) and **storage** (cached blocks, broadcasts). On top sits off-heap **memory overhead** for Python workers and network buffers.

Executor memory model — where the heap goes

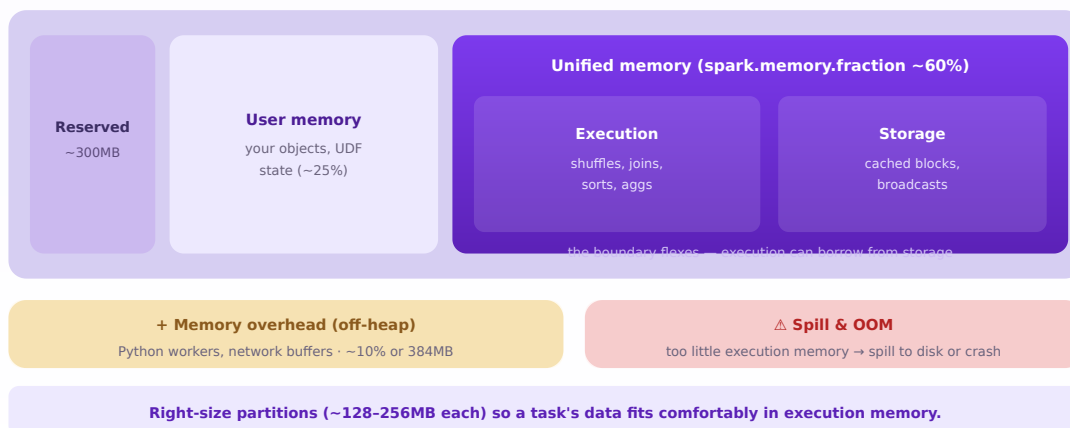


Figure 5.1. An executor's heap is split into reserved, user and a unified region shared by execution and storage, plus off-heap overhead. Starving execution memory causes spill or out-of-memory errors.

📌 Execution and storage share, but execution wins

The boundary between execution and storage is soft — each can borrow the other's free space. But execution can **evict** cached blocks when it needs room, which is why aggressive caching (Module 3) can quietly hurt a shuffle-heavy job.

5.2 Spill and out-of-memory

Spill happens when a task's data exceeds execution memory and Spark writes it to disk — slow, but it survives. An **out-of-memory** error kills the executor outright. Both usually mean the same thing: **partitions are too big**. The fixes, in order: increase the number of partitions so each is smaller; reduce skew (Module 4); cache less; and only then add memory per executor.

 More memory is the last lever, not the first

Raising `spark.executor.memory` masks the symptom and costs money. Smaller, balanced partitions fix the cause. Reach for more memory only after partitioning and skew are sorted.

5.3 Allocating executors, cores & partitions

Three numbers interact. **Cores per executor**: around **4-5** balances parallelism against I/O contention (very high core counts hurt throughput). **Memory per executor**: enough that a partition fits with room for execution. **Partition count**: aim for **2-4× the total cores** so every slot stays busy, with each partition around 128-256 MB.

PYSPARK / PYTHON

```
# Sensible defaults for a SkyOps batch job (illustrative)
spark.conf.set("spark.sql.shuffle.partitions", "400") # ~ 2-4× total cores
spark.conf.set("spark.sql.adaptive.enabled", "true") # let AQE coalesce
# executor shape (set on the cluster, shown for context):
# spark.executor.cores = 4
# spark.executor.memory = 16g
# spark.executor.memoryOverhead = 2g # headroom for Python / Arrow

# Check what you actually got:
print(spark.sparkContext.defaultParallelism)
```

CONSOLE OUTPUT

128

5.4 Cluster configuration & autoscaling

A cluster is a **driver** plus **executors**. Keep the driver modest unless you `collect()` large results to it. Size the executors for the data, and enable **autoscaling** so the cluster adds workers under load and releases them when idle — performance when you need it, cost savings when you do not.

Cluster shape & autoscaling — match resources to the workload

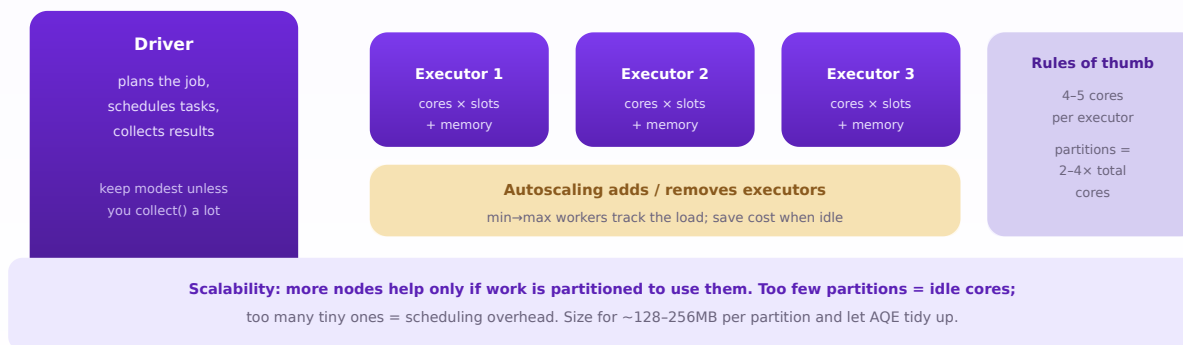


Figure 5.2. Cluster performance comes from matching executors, cores and partitions to the workload, with autoscaling adding capacity only when the load needs it.

💡 Pick the right instance family

Shuffle- and cache-heavy jobs want **memory-optimised** instances; compute-bound transforms want **compute-optimised**. For SkyOps batch ETL we use memory-optimised workers with autoscaling between a small floor and a capped ceiling.

5.5 Scalability considerations

More nodes help only if the work is **partitioned to use them**. With too few partitions, extra cores sit idle; with too many tiny ones, scheduling overhead dominates. As the SkyOps fleet and history grow, scale **partitions with the data** (keep ~128–256 MB each) and let autoscaling add executors — rather than guessing a fixed huge cluster up front.

🔧 Lab 5.1 — Cure a spill

Find a SkyOps stage in the Spark UI that reports spill. (1) Note its partition count and the spill size. (2) Increase `spark.sql.shuffle.partitions` (or repartition) so partitions are smaller, and re-run. (3) Confirm spill drops toward zero and the stage speeds up. (4) Only if spill persists, reason about whether skew (Module 4) or memory per executor is the real cause.

🔑 Key takeaways

- An executor's heap = reserved + user + a unified execution/storage region, plus off-heap overhead
- Spill and OOM almost always mean partitions are too big — make them smaller first
- Use ~4–5 cores per executor and ~2–4 × cores in partitions, each ~128–256MB
- Add executor memory last, after partitioning and skew are addressed
- Right-size the driver, choose the instance family for the workload, and enable autoscaling
- Scale partitions with the data so added nodes are actually used

06

VECTORISE

Integrating Optimized Libraries & UDFs

Plain Python UDFs throw away Spark's speed. Built-in functions, Pandas UDFs and vectorized libraries get it back.

Integrating Optimized Libraries & UDFs

Everything in Modules 1–5 keeps work inside Spark’s fast, code-generated engine. The fastest way to undo it is a row-by-row Python UDF. This module shows how to add Python logic *without* paying that price.

🎯 Learning objectives

- Explain why plain Python UDFs are slow
- Prefer built-in functions wherever possible
- Use vectorized Pandas UDFs with Apache Arrow
- Apply NumPy/Pandas and pandas-API-on-Spark at scale
- Run ML libraries efficiently on Spark
- Choose the right tool in the right order

6.1 Why plain Python UDFs are slow

A regular Python UDF forces Spark to **serialise every row** out of the JVM to a Python worker, run your function one row at a time, and serialise the result back. That breaks whole-stage codegen (Module 1), adds huge overhead, and cannot be optimised by Catalyst. On big data it can be the slowest part of the whole job.

6.2 Built-in functions first, then Pandas UDFs

Before writing any UDF, check `pyspark.sql.functions` — most logic (string ops, dates, math, `when` / `otherwise`, windowing) already exists as a native, optimised function. When you genuinely need custom Python, use a **Pandas UDF**: it moves whole columns as **Arrow** batches and runs your **vectorised** NumPy/pandas code per batch — 10–100× faster than a row-at-a-time UDF.

Python UDF vs Pandas UDF — stop paying per row

Plain Python UDF

row → pickle → Python → pickle → row

serialise every single row to a Python worker and back — huge overhead, no JVM speed

slow X · breaks vectorisation

Pandas UDF (vectorised, Arrow)

batch → Arrow → pandas Series → batch

whole columns move as Arrow batches; NumPy/pandas vectorised maths on each batch

fast ✓ · 10–100× over plain UDF

Order of preference: built-in functions > pandas UDF > plain Python UDF (last resort).

Pandas-API-on-Spark and MLlib give the same vectorised speed for analysis and ML at scale.

Figure 6.1. Plain Python UDFs serialise every row to a Python worker; Pandas UDFs move whole columns as Arrow batches for vectorised, near-JVM speed.

```
import pandas as pd
from pyspark.sql.functions import col, when, pandas_udf

# 1) Best: a built-in expression – no UDF at all
fleet = fleet.withColumn("overheat",
                        when(col("engine_temp_c") > 650, 1).otherwise(0))

# 2) When you must use Python: a vectorised Pandas UDF (Arrow)
@pandas_udf("double")
def fuel_efficiency(fuel: pd.Series, alt: pd.Series) -> pd.Series:
    # vectorised NumPy/pandas maths on whole batches, not per row
    return fuel / (alt / 1000.0)

fleet = fleet.withColumn("fuel_per_1000ft",
                        fuel_efficiency(col("fuel_tph"), col("altitude_ft")))
```

💡 Turn Arrow on

Pandas UDFs rely on Apache Arrow for the JVM↔Python transfer. Keep

`spark.sql.execution.arrow.pyspark.enabled=true` so the batches move efficiently.

6.3 Pandas & NumPy at scale

If your team thinks in pandas, the **pandas API on Spark** runs familiar pandas code across the cluster with the same vectorised speed — the mental model from single-machine analysis carries straight over. Inside Pandas UDFs, lean on **NumPy** vectorisation and broadcasting rather than Python loops; that is what makes each Arrow batch fast.

```
import pyspark.pandas as ps      # pandas API on Spark

psdf = ps.read_parquet("silver/fleet_clean.parquet")  # a distributed DataFrame
summary = (psdf.groupby("type")["fuel_tph"].mean()    # pandas syntax,
           .sort_values(ascending=False))             # Spark execution
print(summary.head())
```

6.4 ML libraries efficiently

For distributed training, **Spark MLlib** operates natively on DataFrames. To apply an existing single-node model (scikit-learn, XGBoost) across a huge dataset, wrap inference in a **Pandas UDF** so each executor scores Arrow batches in parallel — you keep your familiar library and still scale out.

✈️ SkyOps inference at fleet scale

Our fuel-anomaly model is a scikit-learn estimator. Loading it once per executor and scoring inside a Pandas UDF lets us run it over months of telemetry in minutes — no rewrite, full parallelism, vectorised batches.

6.5 The right tool, in the right order

Preference	Tool	Why
1st	Built-in SQL functions	Native, optimised, code-generated
2nd	Pandas UDF (Arrow, vectorised)	Custom Python at near-JVM speed
3rd	pandas API on Spark / MLlib	Familiar APIs, distributed execution
Last	Plain Python UDF	Only when nothing above works

Lab 6.1 — Replace a slow UDF

Find (or write) a SkyOps transform that uses a plain Python UDF. (1) Time it and note in the Spark UI that the stage is Python-bound. (2) Rewrite it first as a built-in expression if possible; otherwise as a vectorised `@pandas_udf`. (3) Re-run and compare the stage time. Record the speed-up — it is usually dramatic.

Key takeaways

- Plain Python UDFs serialise every row and break codegen — avoid them
- Reach for built-in functions first; they are native and optimised
- Pandas UDFs move Arrow batches and run vectorised NumPy/pandas — 10-100× faster than row UDFs
- pandas API on Spark scales familiar pandas code; MLlib trains natively on DataFrames
- Apply single-node models (sklearn/XGBoost) at scale via Pandas UDF inference
- Order of preference: built-in ► Pandas UDF ► pandas-API/MLlib ► plain Python UDF